



CPU EDUCATION

# COMP90051

## 期末班

□ 2 讲解

TUTOR: JJ 学长

时长: 2 小时

### 本周内容预览:

1.SVM+习题练习

大+小可能

2.Kernel 的定义及证明

大+小可能

3.迭代优化

小

4.Perception

小



CPU EDUCATION



## 关于CPU EDUCATION

CPU Education成立于2018年，总部在澳大利亚墨尔本，目前国内办公室在广州。经过近多年的积累与发展，公司凭借优质的教学质量和专业的研发团队力量不断壮大，成为全球最受学生欢迎的教育机构之一，在众多学生及老师中享有良好的口碑。CPU Education争做行业标杆，为不同阶段的学生提供专业，专属，专注的教学服务。

致力于为学生实现人人H1的目标，以“为学生提供极具价值的教学服务，培训各行业精英”为宗旨。秉承“教育高于一切”的经营理念，本着服务定制化、师资专业化、教学规范化的要求，打造墨尔本教育行业的“黄埔军校”。

课后，如果您有任何建议和意见，我们都非常欢迎您联系小助手分享您的想法，给予我们改进和提高的机会！感谢您参与CPU Education的辅导课程！



课程反馈



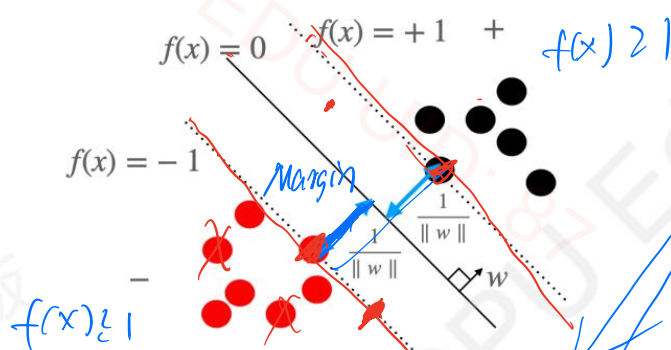
## PART 1 SVM

SVM: hard-margin SVM

找到超平面完美地分类 2 个点

Primal 问题

objective



$$\min_w \frac{\|w\|^2}{2}$$

$$\text{s.t. } 1 - y^{(i)}(w^T x^{(i)} + b) \leq 0,$$

$$i = 1, \dots, n \text{ (data points)}$$

$$y_i (w^T x_i + b) \geq 1$$

$$\max \frac{2}{\|w\|}$$

☆☆

只有 support vector 上的点会影响决策边界

$$\text{loss} = \begin{cases} 0, & \text{if support vector on} \\ +\infty, & \text{else} \end{cases}$$

SVM: hard-margin SVM, dual problem 对 stationary 条件, 消去 w

$$\arg\max_{\lambda} \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j x_i' x_j$$

$$\text{s.t. } \lambda_i \geq 0 \text{ and } \sum_{i=1}^n \lambda_i y_i = 0$$

predict

$$\lambda_i = 0, \text{ 对 } w \text{ 无贡献}$$

$$\lambda_i > 0, \text{ 必在 } \pm 1$$

$$s = b^* + \sum_{i=1}^n \lambda_i^* y_i x_i' x$$

$$a_i^*$$

$$b^*$$

$$y_i (w^T x_i + b) = 1$$

☆ SVM 预测速度仅取决于支持向量数

n = 100万

边界 40f







## SVM: hard-margin SVM

## KKT conditions in hard-margin SVM

$$L(w, b, \lambda) = \frac{\|w\|^2}{2} + \sum_{i=1}^n \lambda_i (1 - y^{(i)}(w^T x^{(i)} + b)),$$

( $i = 1, \dots, n$  data points)

(Primal feasibility)  $g_i(w, b) = 1 - y^{(i)}(w^T x^{(i)} + b) \leq 0$

(Dual feasibility)  $\lambda_i \geq 0$

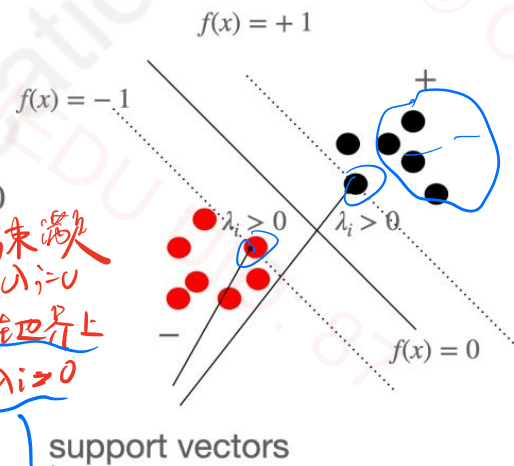
(Complementary slackness)

$$\lambda_i g_i(w, b) = 0$$

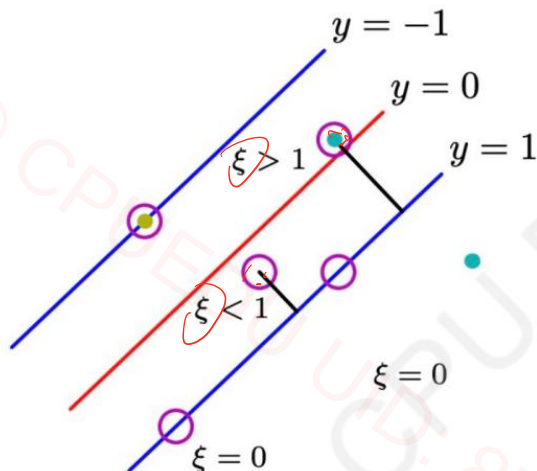
(Stationarity)

$$\frac{\partial L}{\partial w_j} = 0 : w_j = \sum_{i=1}^n \lambda_i y^{(i)} x_j^{(i)}$$

$$\frac{\partial L}{\partial b} = 0 : \sum_{i=1}^n \lambda_i y^{(i)} = 0$$



## SVM: soft-margin SVM



objective

$$\min_w \left( \frac{\|w\|^2}{2} + C \sum_{i=1}^n \xi_i \right)$$

s.t.

$$y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i,$$

$$\xi_i \geq 0 \quad (i = 1, \dots, n \text{ data points})$$







SVM: soft-margin SVM, dual problem

$$\operatorname{argmax}_{\lambda} \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j x_i' x_j$$

弱对偶性

Dual optimum  $D^* \leq$  primal  $P^*$ 

强对偶性

$$D^* = P^*$$

$$\text{s.t. } C \geq \lambda_i \geq 0 \text{ and } \sum_{i=1}^n \lambda_i y_i = 0$$

$$\lambda_i = C$$

在决策边界另一边

$$0 < \lambda_i < C$$

在间隔上

$$\lambda_i = 0$$

无约束

(convex并且存在严格满足约束可行解)

SVM: soft-margin SVM

**KKT conditions in soft-margin SVM**

$$L(w, b, \lambda, \beta, \xi) = \frac{\|w\|^2}{2} + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \lambda_i g_i(w, b, \xi) + \sum_{i=1}^n \beta_i (-\xi_i)$$

(i = 1, ..., n data points)

$$\text{(Primal feasibility)} \quad g_i(w, b, \xi) = 1 - \xi_i - y^{(i)}(w^T x^{(i)} + b) \leq 0,$$

$$-\xi_i \leq 0$$

(Dual feasibility)

$$\lambda_i \geq 0, \beta_i \geq 0$$

都是 lagrange 乘子

(Complementary slackness)

$$\lambda_i g_i(w, b, \xi) = 0 \quad \beta_i \xi_i = 0$$

(Stationarity)

$$\frac{\partial L}{\partial w_j} = 0: \quad w_j = \sum_{i=1}^n \lambda_i y^{(i)} x_j^{(i)}$$

$$\frac{\partial L}{\partial b} = 0: \quad \sum_{i=1}^n \lambda_i y^{(i)} = 0$$

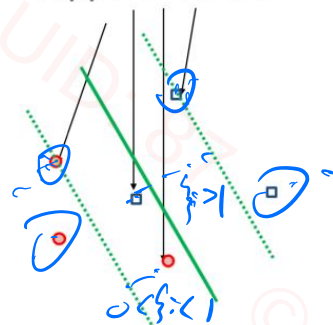
$$\frac{\partial L}{\partial \xi_i} = 0: \quad C - \lambda_i - \beta_i = 0$$

$$\lambda_i = C - \beta_i$$

$$\text{因为 } \beta_i \geq 0$$

$$\text{所以 } 0 \leq \lambda_i \leq C$$

support vectors



☆☆☆☆

无约束分类

$$g_i < 0, \xi_i = 0, \lambda_i = 0$$

不参与决策

刚好在+1, -1超平面

$$g_i = 0, \xi_i = 0$$

$$0 < \lambda_i < C$$

支持向量

$$y_i(w^T x_i + b) < 1, \xi_i > 0$$

$$\beta_i = 0, \lambda_i = C$$

$$\lambda = C$$





2. For the hard-margin support vector machine data points that are support vectors have a margin of 1 from the decision boundary. Explain how this can be the case even when support vectors are further than 1 unit away from the decision boundary in Euclidean space. [4 marks]

在hard-margin SVM中, 1 unit interval 指的是函数间隔即  $y_i(w^T x_i + b) = 1$ . 但实际上的 Euclidean distance 是  $\frac{1}{\|w\|}$ , 所以超过 1 个单位长度, 仍是满足约束

- (h) Suppose you have trained a soft-margin SVM with a RBF kernel, but the performance is poor on both training and validation sets. How will you change the hyperparameters of the SVM to improve the performance? List two strategies. [5 marks]

① 增大  $C$ , 加大对分错的惩罚

② 增大  $\gamma$ , 使得 Kernel 的影响域更小, 更加 focus

3. For a support vector machine with a quadratic kernel, in what situation, if any, would it be better to use the primal program instead of the dual program for training? [4 marks]

Primal 变量维度是  $d$

对偶 变量维度是  $n$

Kernel Matrix  $[n \times n]$  维

数据  $n$  特别大并且  $d$  特别小时

因为对偶问题要计算 Kernel Matrix  
此时  $n \times n$  很大





- (a) Suppose you have trained a *soft-margin support vector machine (SVM)* with a *RBf kernel*, and the performance is very good on the training set while very poor on the validation set. How will you change the hyperparameters of the SVM to improve the performance of the validation set? List two strategies. [5 marks]

①  $C$  减小

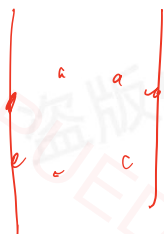
②  $\gamma$  减小

- (e) In what way is the *objective function* for *soft-margin SVM* a relaxation if the *hard-margin SVM* objective? [1 mark]

(slack variable)

It introduce  $\xi_i$  to allow  $y_i(w^T x_i + b) < 1$

- (c) How are *support vectors* characterised by *dual variables*  $\lambda_i$  in the *soft-margin SVM*? (See the formula sheet if needing to refer to the dual formulation.) [1 mark]



$0 < \lambda_i < C$  : 边界上的点

$\lambda_i = C$  : 在间隔内的点

$0 < \lambda_i \leq C$  的点

- (f) *Weak duality* guarantees that a *primal optimum* always upper bounds a *dual optimum*. How can we tell that the *support vector machine's primal and dual optima* are always equal? [5 marks]

因为 SVM 是 convex with linear inequality constraints

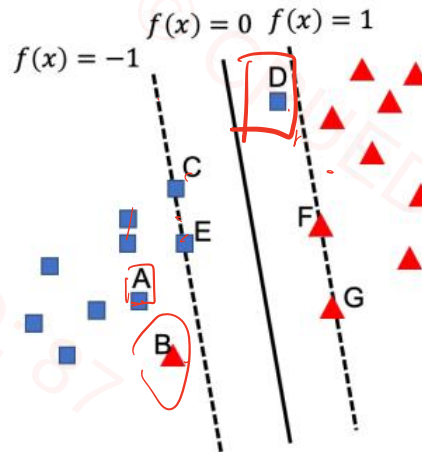
并且存在严格可行解







Assume that the following figure is an illustration of the *decision boundary* learned by a *linear soft-margin SVM* on a two-class data set, with points from one class labelled with blue squares and points from the other class labelled with red triangles.



The *Lagrangian function* of the *soft-margin SVM* is:

$$L(w, b, \lambda, \beta, \xi) = \frac{\|w\|^2}{2} + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \lambda_i g_i(w, b, \xi) + \sum_{i=1}^n \beta_i (-\xi_i)$$

$$g_i(w, b, \xi) = 1 - \xi_i - y^{(i)}(w^T x^{(i)} + b)$$

where  $f(x) = w^T x + b$ ,  $\lambda_i$ ,  $\beta_i$  are *Lagrange multipliers* and  $\xi_i$  is the *slack variable* of the  $i^{th}$  data point,  $C$  is the *slack penalty*.

Given the seven points A, B, C, D, E, F, and G as shown in the figure,

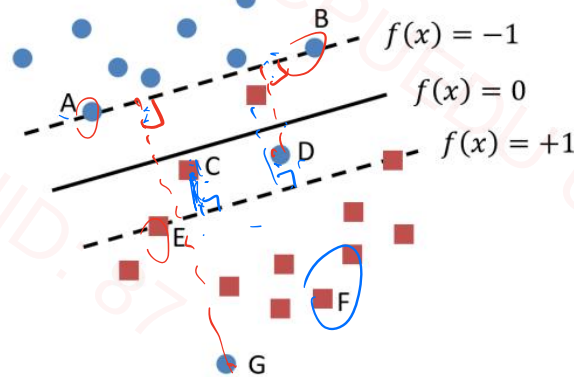
- Identify the data point(s) with  $\lambda_i = C$  (by index A-G. Note: wrong points in will be deducted marks). [4 marks]  
B, D
- Identify the data point(s) with  $\xi_i = 0$  (by index A-G. Note: wrong points in will be deducted marks). [4 marks]  
A, C, E, F, G
- If Point A is removed from the data set, will the decision boundary change? Explain why. [4 marks]

No because A is <sup>neither</sup> support vector nor the  $\xi_i > 0$



**Question 4: Support Vector Machines [16 marks]**

Assume that the following figure is an illustration of the *decision boundary* learned by a *linear soft-margin SVM* on a two-class data set, with points from one class labelled with blue circles and points from the other class labelled with red squares.



The *Lagrangian function* of the *soft-margin SVM* is:

$$L(w, b, \lambda, \beta, \xi) = \frac{\|w\|^2}{2} + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \lambda_i g_i(w, b, \xi) + \sum_{i=1}^n \beta_i (-\xi_i)$$

where

$$g_i(w, b, \xi) = 1 - \xi_i - y^{(i)}(w^T x^{(i)} + b)$$

and  $f(x) = w^T x + b$ . Here  $\lambda_i$ ,  $\beta_i$  are *Lagrange multipliers* and  $\xi_i$  is the *slack variable* of the  $i^{th}$  data point,  $C$  is the *slack penalty*.

Given the seven points A, B, C, D, E, F, and G as shown in the figure,

- Identify the training point(s) with  $\lambda_i = 0$  (by index A–G). [4 marks]
- Identify the training point(s) with  $\xi_i \neq 0$  (by index A–G). [6 marks]
- For each training point identified in the previous part, is  $\xi_i < 0$ ,  $0 < \xi_i < 1$ ,  $1 < \xi_i < 2$ , or is  $\xi_i > 2$ ? [6 marks]

$$F: \xi_i = 0$$

$$C: 0 < \xi_i < 1$$

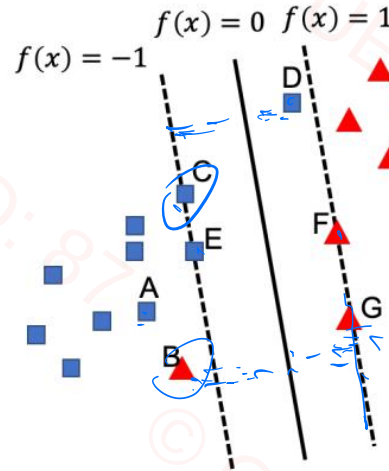
$$D: 1 < \xi_i < 2$$

$$G: \xi_i > 2$$



**Question 2: Support Vector Machines [14 marks]**

Assume that the following figure is an illustration of the *decision boundary* learned by a *linear soft-margin SVM* on a two-class data set, with points from one class labelled with blue squares and points from the other class labelled with red triangles.



- (a) Identify the training points that are *support vectors* (by index A–G). [6 marks]
- (b) The *Lagrangian function* of the *soft-margin SVM* is:

$$L(w, b, \lambda, \beta, \xi) = \frac{\|w\|^2}{2} + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \lambda_i g_i(w, b, \xi) + \sum_{i=1}^n \beta_i (-\xi_i)$$

$$g_i(w, b, \xi) = 1 - \xi_i - y^{(i)}(w^T x^{(i)} + b)$$

For each of the four points A, B, C and D, is  $\lambda_i = 0$ ,  $0 < \lambda_i < C$ ,  $\lambda_i = C$  or  $\lambda_i > C$ ? [4 marks]

For each of the four points A, B, C and D, is  $\xi_i = 0$ ,  $0 < \xi_i < 1$ ,  $1 < \xi_i < 2$  or  $\xi_i > 2$ ? [4 marks]

$$\begin{array}{ll} A: & \lambda_i = 0 \quad \xi_i = 0 \\ B: & \lambda_i = C \quad \xi_i = 2 \\ C: & 0 < \lambda_i < C \quad \xi_i = 0 \\ D: & \lambda_i = C \quad 1 < \xi_i < 2 \end{array}$$







## PART 2 Kernel

### Hard-margin SVM Dual Problem

#### Dual function

$$\theta_d(\lambda) = \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{k=1}^n \lambda_i \lambda_k y^{(i)} y^{(k)} (x^{(i)})^T x^{(k)}$$

dot product

可以kernelize

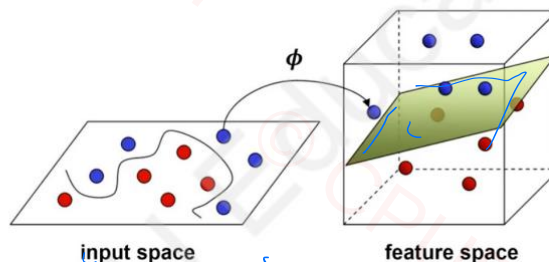
#### Dual problem

$$\begin{aligned} \max_{\lambda} \quad & \theta_d(\lambda) \\ \text{s.t.} \quad & \lambda_i \geq 0 \quad \text{and} \quad \sum_{i=1}^n \lambda_i y^{(i)} = 0 \end{aligned}$$

$$\phi(x^{(i)})^T \phi(x^{(k)})$$

### What is kernel?

- If data is not linearly separable, we can transform original data points to a new space where data is linearly separable (hopefully, not guaranteed)



#### steps

- find a proper feature transformation function  $\phi$  (may not be easy to find)
- transform each data point  $x$  to  $\phi(x)$  (can be slow if  $\phi(x)$  is in high dimension)
- replace original dot product of feature pairs with transformed feature pairs in the objective of hard-margin dual problem

Explicit 计算核函数

$$(x^{(i)}, x^{(j)}) \rightarrow \phi(x^{(i)})^T \phi(x^{(j)})$$





### What is kernel?

- kernel  $K(x, y)$  is a function that is equivalent to a dot product between two vectors in "some" space.
- Use kernel instead of explicitly transforming feature vectors one by one and applying dot products
- Advantages of using kernel
  1. don't need to find  $\phi$
  2. don't need to transform feature vectors
  3. don't need to do dot product

### Commonly Used kernels

- Polynomial kernel

$$K(u, v) = (u^T v + c)^d \quad (c \geq 0, d \geq 0)$$

- Radial Basis Function kernel (RBF kernel)

$$K(u, v) = \exp(-\gamma \|u - v\|^2)$$

small  $\gamma$ : tend to underfitting  
large  $\gamma$ : tend to overfitting





Prove a pairwise function is kernel

① 直接找到  $\phi$ 

- method 1: Find  $\phi$  directly
- method 2: Mercer's Theorem
- method 3: use rules

- $K(u, v) = K_1(u, v) + K_2(u, v)$
- $K(u, v) = cK_1(u, v)$  [ $c > 0$ ]
- $K(u, v) = f(u)K_1(u, v)f(v)$  [ $f(x)$  is a real-valued function]

$$K(u, v) = \phi(u)^T \phi(v)$$

$$\text{e.g. } K(u, v) = (u^T v + 1)^2$$

$$K(u, v) = [u_1^2, \sqrt{2}u_1u_2, u_2^2, \sqrt{2}u_1, \sqrt{2}u_2]^T$$

$$[v_1^2, \sqrt{2}v_1v_2, v_2^2, \sqrt{2}v_1, \sqrt{2}v_2]$$

$$\text{let } u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \quad v = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

$$(u_1v_1 + u_2v_2 + 1)^2 = \mathbb{1}^T \mathbb{1}$$

$$K(u, v) = k(v, u)$$

$$\text{且 } \forall C \in \mathbb{R}^n \quad C^T K C \geq 0$$

$$|C^T K C| \geq 0 \quad |C^T K(x_i, x_j)|$$

e.g. B, C 都是 PSD

$$B \otimes C \Rightarrow \text{PSD}$$

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \otimes \begin{bmatrix} 2 & 2 \\ 3 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 6 \\ 12 & 15 \end{bmatrix}$$

Show the following pairwise functions are valid kernels

$$\kappa'(x, x') = \kappa_\alpha(x, x')\kappa_\beta(x, x') \text{ for valid kernels } \kappa_\alpha, \kappa_\beta$$

$$\kappa'(x, x') = \exp(\kappa(x, x')) \text{ for valid kernel } \kappa$$

$$\kappa(1,1) \times \kappa(1,1)$$

$$\textcircled{2} \text{ 推广: } \text{令 } y = k(x, x')$$

$$e^y = \sum_{i=0}^{\infty} \frac{y^i}{i!}$$

$$= \frac{y^0}{0!} + \frac{y^1}{1!} + \frac{y^2}{2!} + \dots$$

因为  $y = k(x, x')$  都是 valid kernel

if  $k_1, k_2$  是 valid kernel

- $k_1 + k_2$  是 valid kernel
- $\lambda k_1$  是 valid kernel
- $k_1 \cdot k_2$  还是 kernel

$$\text{则 } \frac{y^i}{i!} \text{ 是 valid kernel}$$

所以它们的和也是 valid kernel

① 证明  $K'$  是 PSD $\kappa_\alpha$  是 valid kernel,  $K_\alpha$  是 PSD $\kappa_\beta$  是 valid kernel,  $K_\beta$  是 PSD

$$K' = K_\alpha \otimes K_\beta$$

$$\text{因为自内元素 } \kappa'(x, x') = \kappa_\alpha(x, x')\kappa_\beta(x, x')$$

用两个 PSD 的 element-wise product 是 PSD

 $K'$  是 PSD $\kappa'(x, x')$  是 kernel





## Question 4: Kernels [10 marks]

Consider a valid kernel  $k(\mathbf{u}, \mathbf{v})$  and a constant  $c$ , where  $\mathbf{u}$  and  $\mathbf{v}$  are vectors with dimension  $d_1$  and  $d_2$ .  
Prove that their sum  $f(\mathbf{u}, \mathbf{v}) = k(\mathbf{u}, \mathbf{v}) + c$  is also a valid kernel. ( $c > 0$ )

① 因为  $k(u, v)$  是 valid

$$k(u, v) = \phi(u)^T \phi(v)$$

$$\phi(u)^T \phi(v) = \phi(u)^T \phi(v) + c = k(u, v) + c = f(u, v)$$

$$\phi(x) = \begin{bmatrix} \phi(x) \\ \sqrt{c} \end{bmatrix} \quad (c > 0)$$

## Question 15: Kernels [12 marks]

If  $k_1(\mathbf{a}, \mathbf{a}')$  and  $k_2(\mathbf{b}, \mathbf{b}')$  are both valid kernels over vector valued inputs of size  $d_1$  and  $d_2$  respectively, prove that

$$k_3 \left( \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix}, \begin{bmatrix} \mathbf{a}' \\ \mathbf{b}' \end{bmatrix} \right) = k_1(\mathbf{a}, \mathbf{a}') + k_2(\mathbf{b}, \mathbf{b}') = \phi_1(\mathbf{a})^T \phi_1(\mathbf{a}') + \phi_2(\mathbf{b})^T \phi_2(\mathbf{b}')$$

is also a valid kernel over vectors of size  $d_1 + d_2$ . Note that the symbol ' means 'prime' not vector transpose,

and  $\begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix}$  denotes concatenation of the two vectors.

$k_1, k_2$  是 valid

存在两个  $\phi, \phi_2$  s.t.

$$k_1(\mathbf{a}, \mathbf{a}') = \phi_1(\mathbf{a})^T \phi_1(\mathbf{a}')$$

$$k_2(\mathbf{b}, \mathbf{b}') = \phi_2(\mathbf{b})^T \phi_2(\mathbf{b}')$$

> 内积

定义新的 特征映射

$$\Phi \left( \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix} \right) = \begin{bmatrix} \phi_1(\mathbf{a}) \\ \phi_2(\mathbf{b}) \end{bmatrix}$$

$$\langle \Phi \left( \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix} \right), \Phi \left( \begin{bmatrix} \mathbf{a}' \\ \mathbf{b}' \end{bmatrix} \right) \rangle = \left\langle \begin{bmatrix} \phi_1(\mathbf{a}) \\ \phi_2(\mathbf{b}) \end{bmatrix}, \begin{bmatrix} \phi_1(\mathbf{a}') \\ \phi_2(\mathbf{b}') \end{bmatrix} \right\rangle$$

$$= \phi_1(\mathbf{a})^T \phi_1(\mathbf{a}') + \phi_2(\mathbf{b})^T \phi_2(\mathbf{b}') = k_1(\mathbf{a}, \mathbf{a}') + k_2(\mathbf{b}, \mathbf{b}')$$





## Question 3: Kernel machines [16 marks]

- (a) The *soft-margin support vector machine* imposes box constraints on the *dual parameters* in training, namely  $0 \leq \lambda_i \leq C$ . Explain how reducing the value of  $C$  has the effect of increasing the *margin of separation* of the training data. You should assume a *linear kernel*, and recall the margin can be expressed as  $\frac{1}{\|\mathbf{w}\|}$  where  $\mathbf{w}$  are the *primal weights*. [8 marks]
- (b) We wish to use the function  $k(\mathbf{u}, \mathbf{v}) = \mathbf{u}^T \mathbf{B} \mathbf{v}$  as a kernel, where  $\mathbf{B}$  is a  $d \times d$  matrix and  $\mathbf{u}, \mathbf{v}$  are both  $d$  dimensional vectors. Is this a valid kernel? Do any conditions need to be placed on  $\mathbf{B}$  for this to be the case? Present a mathematical argument to support your answer. [8 marks]

a). Primal  $\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum \xi_i$

减小  $C$ , 就更看重  $\frac{1}{2} \|\mathbf{w}\|^2$ , 优化器愿意牺牲  $\xi_i$  来换取更小的  $\|\mathbf{w}\|$ , 倾向于增大  $\frac{1}{\|\mathbf{w}\|}$

b). 猜对称半正定  
 $\mathbf{B}$  是 PSD  $\mathbf{B} = \mathbf{L} \mathbf{D} \mathbf{L}^T$   $\mathbf{D}$  是 Diagonal matrix

$$\begin{aligned} k(\mathbf{u}, \mathbf{v}) &= \mathbf{u}^T \mathbf{B} \mathbf{v} \\ &= \mathbf{u}^T \mathbf{L} \mathbf{D} \mathbf{L}^T \mathbf{v} \\ &= (\mathbf{u}^T \mathbf{L} \mathbf{D}) (\mathbf{L}^T \mathbf{v}) \\ &= (\mathbf{D}^T \mathbf{L}^T \mathbf{u})^T (\mathbf{D} \mathbf{L}^T \mathbf{v}) \end{aligned}$$

$$\mathbf{D}(\mathbf{x}) = \mathbf{D} \mathbf{L}^T \mathbf{x}$$

证毕

- (b) Consider a *polynomial kernel* of degree  $p = 12$  (you may refer to the formula sheet). Explain a benefit of using the *kernel trick* when using this kernel in the *SVM* vs explicit basis expansion. [1 mark]

Simple computation

don't need to explicitly calculate

dot-product





## PART 3 迭代优化复习

### Iterative Optimization: Gradient Descent

objective

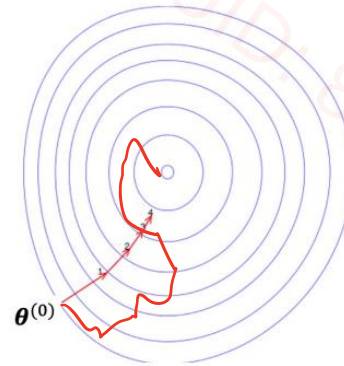
$$\hat{\theta} \in \underset{\theta \in \Theta}{\operatorname{argmin}} L(\text{data}, \theta)$$

整批 Data 训练

steps

1. Choose  $\theta^{(1)}$  and some  $T$
2. For  $i$  from 1 to  $T^*$ 
  1.  $\theta^{(i+1)} = \theta^{(i)} - \eta \nabla L(\theta^{(i)})$
3. Return  $\hat{\theta} \approx \theta^{(i)}$

init



SGD: 每次只用一个样本或一小批样本

### Iterative Optimization: Gradient Descent

Notes:

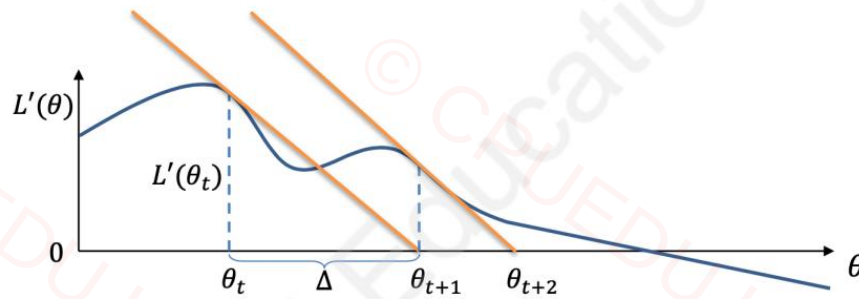
- if objective function is convex (bowl shaped), GD is guaranteed to find a unique global minimum.
- we can use a subset of training set to estimate the gradient vector (mini-batch gradient descent)
  - the size of the subset is called batch size
  - if batch size = 1, we call it stochastic gradient descent







## Iterative Optimization: Newton-Raphson



initialize  $\theta_0$   
repeat

$$\theta_{t+1} = \theta_t - L'(\theta_t) / L''(\theta_t)$$

vector/matrix format:  $\theta_{t+1} = \theta_t - (\nabla_2 L(\theta_t))^{-1} \nabla L(\theta_t)$

Hessian

## Iterative Optimization: Newton-Raphson

Notes:

- Newton-Raphson (using second-order derivative information) may converge faster than

Gradient Descent

- but not guaranteed faster than GD, Hessian matrix inversion can be expensive





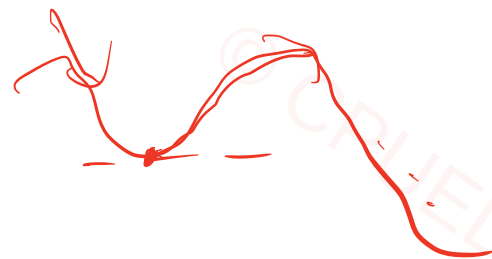
## Momentum

Momentum accumulates velocity to accelerate convergence:

$$v^{(t)} = \alpha v^{(t-1)} + \eta \nabla L(\theta^{(t)})$$

$$\theta^{(t+1)} = \theta^{(t)} - v^{(t)}$$

where  $\alpha$  is the momentum parameter (typically  $\alpha = 0.9$ ).



## ADAGRAD

AdaGrad adapts the learning rate to parameters by scaling inversely with the accumulated squared gradients:

$$g_i^{(t)} = g_i^{(t-1)} + (\Delta L(\theta^{(t)}))_i^2$$

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \frac{\eta}{\sqrt{g_i^{(t)} + \epsilon}} (\Delta L(\theta^{(t)}))_i$$

where  $\epsilon$  is a small constant for numerical stability (e.g.  $10^{-8}$ ).

累计梯度平方

起到的

$g_i^{(t)} \uparrow$

## ADAM

Adam combines momentum and adaptive learning rates:

$$m^{(t)} = \beta_1 m^{(t-1)} + (1 - \beta_1) \Delta L(\theta^{(t)})$$

$$v^{(t)} = \beta_2 v^{(t-1)} + (1 - \beta_2) (\Delta L(\theta^{(t)}))^2$$

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{m^{(t)}}{\sqrt{v^{(t)} + \epsilon}}$$

一阶动量

二阶动量

where typical values are  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ , and  $\epsilon = 10^{-8}$ .





7. Explain why the use of *momentum* in an optimiser can help avoid getting trapped at *local optima* or *saddle points*, with respect to the training of machine learning models with *non-convex training objectives*. [4 marks]

引入了惯性信息，引入历史梯度信息来将前几步也是我的速度上，有助于跳出局部最优 (saddle 鞍点接近于0)

- (h) Explain why only one iteration of *Newton-Raphson* is required to train *linear regression*. [5 marks]

$$L(\beta) = \frac{1}{2} (y - X\beta)^T (y - X\beta)$$

$$\nabla L(\beta) = -X^T (y - X\beta)$$

$$H = X^T X$$

$$\beta^{(t+1)} = \beta^{(t)} - H^{-1} \nabla L(\beta^{(t)})$$

$$\beta^{(t+1)} = \beta^{(t)} - (X^T X)^{-1} [-X^T (y - X\beta^{(t)})]$$

$$= \beta^{(t)} + (X^T X)^{-1} X^T (y - X\beta^{(t)})$$

$$= \beta^{(t)} + (X^T X)^{-1} X^T y - \underbrace{(X^T X)^{-1} X^T X}_{I} \beta^{(t)} = (X^T X)^{-1} X^T y$$

- (a) The *Adagrad* optimiser is a variant of *stochastic gradient descent* with a *dynamic learning rate*. Explain how the dynamic learning rate operates, and why this is important during optimisation of machine learning models. [5 marks]

将自身的梯度平方累加，并将学习率除以累加值的平方根  
一开始时，累加的少， $\eta$ 大，后面累加多， $\eta$ 变小

这样子在前面时  $\eta$  大，鼓励更新  
后面  $\eta$  小，避免速度更小







- (a) How does the value of the *learning rate*,  $\eta$  in *stochastic gradient descent* affect the progress of training? [5 marks]

$$\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \nabla L(\theta^{(t)})$$

$\eta$  正大, 参数可能在最优点附近动荡

过小, 训练速度慢

- (b) Describe something that can go wrong specifically with *gradient descent* while training a learner. Describe one approach to mitigating the problem. [2 marks]

stuck in local minimum

采用 momentum

### Question 6: Gradient Descent [3 marks]

Suppose you are selecting parameter vector  $\theta \in \mathbb{R}^d$  to minimise loss over a training set, with the loss from predicting with  $\theta$  on the  $i$ th example written as  $\ell(\theta, \mathbf{x}_i, y_i)$ , for  $i \in \{1, \dots, n\}$ .

- (a) Write down the *gradient descent algorithm* for accomplishing this task. Explain any additional notation/variables you introduce. [1.5 marks]
- (b) Explain how *stochastic gradient descent* would differ in approximating this problem. [1.5 marks]

a). 
$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(\theta, \mathbf{x}_i, y_i)$$

① Init  $\theta^{(0)}$

② for 1 to T

$$\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \nabla_{\theta} L(\theta^{(t)})$$

$\eta$ : learning rate  $\nabla$ : 梯度

直至迭代到 T 次为止

b). SGD 每次只使用一个样本或  
小批量样本, 计算代价更低



**Question 3: Newton-Raphson for Linear Regression** [10 marks]

Recall that *linear regression*'s objective can be written as  $L(\theta) = \frac{1}{2} \|\mathbf{X}\theta - \mathbf{y}\|_2^2$ , where  $\mathbf{X}$  is a  $n \times d$  matrix of training instances in rows,  $\mathbf{y} \in \mathbb{R}^n$  are the training labels, and  $\theta \in \mathbb{R}^d$  are the parameters to learn by minimising  $L(\theta)$ . Show that for any starting  $\theta_0 \in \mathbb{R}^d$ , a single step of Newton-Raphson optimisation of  $L(\theta)$  recovers the *normal equation* exact solution. (Hints for taking vector/matrix-valued derivatives with respect to  $\theta$ , for constants matrix  $\mathbf{A}$  and vector  $\mathbf{b}$ . Hint 1:  $\nabla(\mathbf{A}\theta + \mathbf{b}) = \mathbf{A}$ . Hint 2:  $\nabla_2(\mathbf{A}\theta) = \mathbf{A}$ . Hint 3:  $\nabla(\|\mathbf{f}(\theta)\|_2^2) = 2(\nabla \mathbf{f}(\theta))' \mathbf{f}(\theta)$ . Where prime  $'$  denotes transpose,  $\nabla$  the gradient, and  $\nabla_2$  the Hessian as in class, here always with respect to  $\theta$ .)

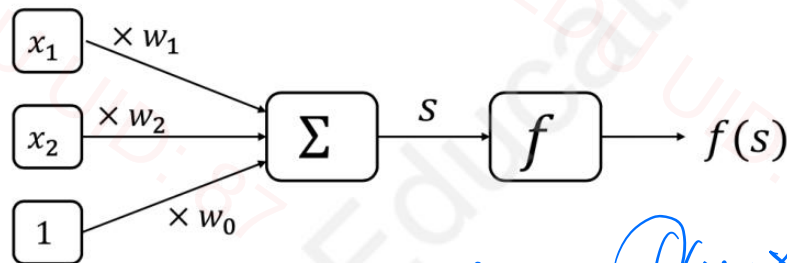
見上面





## PART 4 Perception

Feedforward Neural Networks: Perceptron – a single unit in NN



Linear classifier:  $s$  is a linear function of inputs

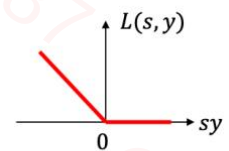
$f(s) = \text{flux}(x_1, x_2, x_3, w_1, w_2, w_3)$   
 Activation function  
 (signal)

Feedforward Neural Networks: Perceptron training

loss function

- \*  $L(s, y) = 0$  if both  $s, y$  have the same sign
- \*  $L(s, y) = |s|$  if both  $s, y$  have different signs

$$L(s, y) = \max(0, -sy)$$



gradient descent

$$\frac{\partial L}{\partial w_i} = -x_i \text{ when } y = 1 \text{ and } s < 0$$

$$\frac{\partial L}{\partial w_i} = x_i \text{ when } y = -1 \text{ and } s > 0$$



If  $y = 1$ , but  $s < 0$

$$w_i \leftarrow w_i + \eta x_i$$

$$w_0 \leftarrow w_0 + \eta$$

If  $y = -1$ , but  $s \geq 0$

$$w_i \leftarrow w_i - \eta x_i$$

$$w_0 \leftarrow w_0 - \eta$$

$$w = w + \eta y_i x_i$$



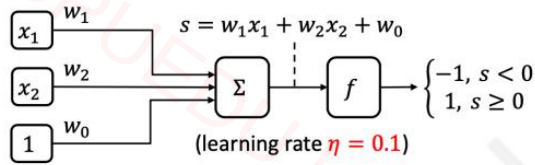




Suppose we initialize the following perceptron with  $w_1=0.2$ ,  $w_2=0.0$ ,  $w_0=-0.1$ . Please update the three parameters using the two data points in order (learning rate = 0.1):

data point 1:  $x_1 = 1$ ,  $x_2 = 1$ ,  $y = -1$

data point 2:  $x_1 = 2$ ,  $x_2 = 1$ ,  $y = 1$



$$-0.1 + 0.2 \times 1 + 1 \times 0 = 0.1 > 0 \\ f(s) = 1$$

$$w_1 = 0.2 - 0.1 \cdot 1 = 0.1$$

$$w_2 = 0 - 0.1 \cdot 1 = -0.1$$

$$w_0 = -0.1 - 0.1 = -0.2$$

$$-0.2 + 0.1 \times 2 + (-0.1) \cdot 1 = -0.1 \\ f = -1$$

$$w_1 = 0.1 + 2 \times 0.1 = 0.3$$

$$w_2 = -0.1 + 1 \times 0.1 = 0$$

$$w_3 = -0.2 + 0.1 = -0.1$$

## Kernelised perceptron training rule

Set  $\alpha = 0$

应用 kernel

$$s_i = \sum_{j=1}^N \alpha_j y_j K(x_j, x_i)$$

For  $t$  from 1 to  $T$  (epochs)

For  $i$  from 1 to  $N$  (training examples)

Compute  $s_i = \sum_{j=1}^N \alpha_j y_j K(x_j, x_i)$

If  $s_i y_i \leq 0$ : (sample  $i$  misclassified)

$$\alpha_i \leftarrow \alpha_i + \eta \quad \text{学习率}$$





(h) List two general strategies for proving that a *learning algorithm* can be *kernelised*. [5 marks]

① SVM

② Perceptron

## Question 2

1 / 1 pts

Imagine you are training a Perceptron as described in lectures. That is, for  $s = w_0 + w_1 x_1 + w_2 x_2$ ,  $f(s)$  predicts positive for  $s \geq 0$  and it predicts negative when  $s < 0$ .

Consider that you are using the Perceptron loss function  $L(s, y) = \max\{0, -sy\}$  as given in lectures, and the standard Perceptron update rule with learning rate  $\eta = 1$ .

If you currently have weights  $(w_0, w_1, w_2) = (2, 1, -3)$  what will the *new weight*  $w_1$  be after processing training example  $(x_1, x_2) = (2, 1), y = -1$ ?

- (a) Consider networks with hundreds of layers of *perceptrons* – *neurons* with linear activation functions that output a constant times the weighted sum of their inputs. Are these deep networks? Explain why. [2 marks]

层数较多，但由于 activation function 是线性的

$$f(x) = w_0 + x$$

等价于 单层线性模型

并没有任何的非线性表达能力









CPU EDUCATION

CPU内部资料  
盗版必究举报有奖



CPU EDUCATION



CPU墨大官方客服

「更多课程服务、资源获取详情  
欢迎滴滴猪猪客服！」

CPU·EDU成长有你陪伴  
· 让海外学习更轻松 ·